

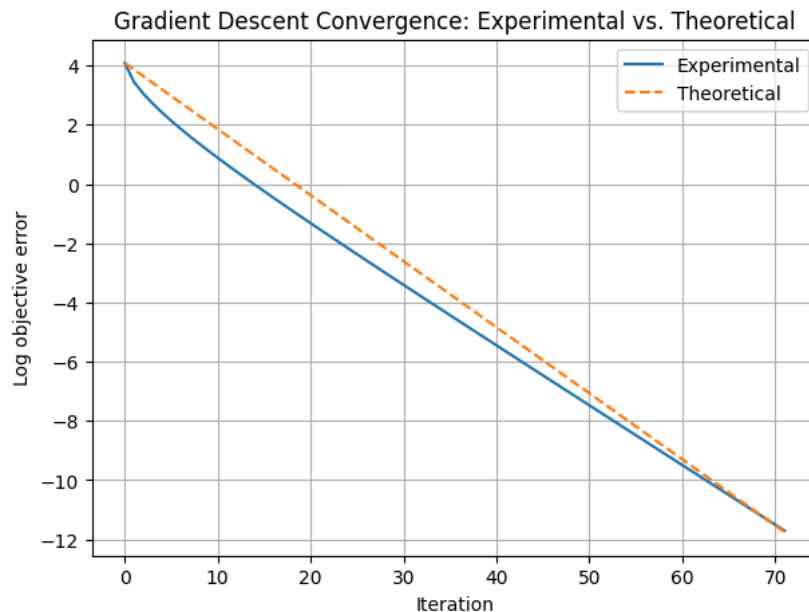
Problem 1

Task 1.

Algorithm: $x_{k+1} = x_k - \alpha(Ax_k - b)$, with $\alpha = \frac{2}{L+\mu}$

L = largest, μ = smallest eigenvalue of A -> ensures stability and good convergence for a strongly convex problem.

We start with $x_0 = 0$. Then update x using α until $\|\nabla f(x)\|$ is below a threshold.

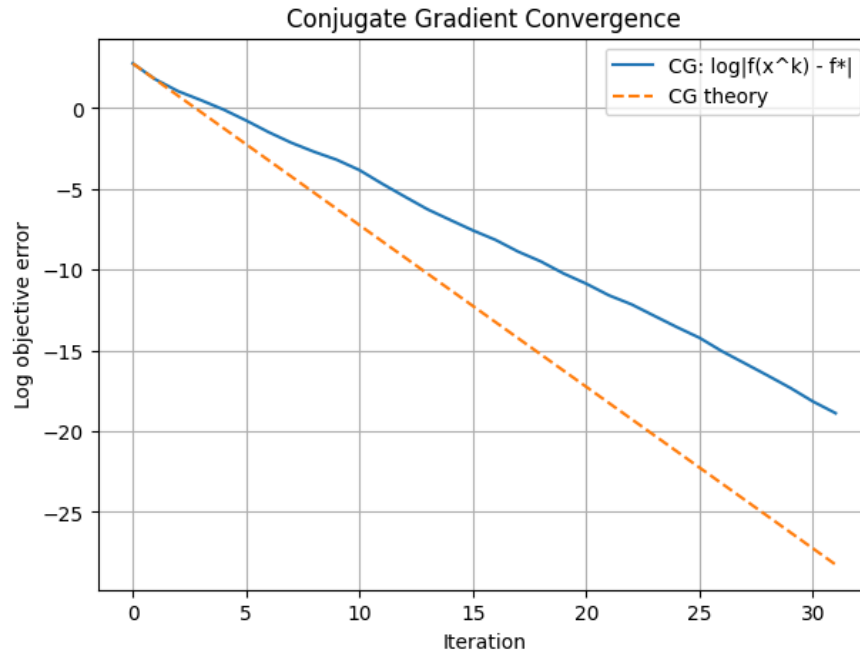


The plotted curve of $\log \log [f(x^t) - f^*]$ vs t . shows almost a linear (straight line) on a log scale, indicating linear convergence. This matches the theoretical result.

Task 2.

- Residual $r_k = b - Ax_k$
- Direction $p_k = r_k + \beta_{k-1} p_{k-1}$
- Step size $\alpha_k = \frac{r_k^T r_k}{p_k^T A p_k}$

Initialize $x_0 = 0$, $r_0 = b$, $p_0 = r_0$ -> Update x , r , p each iteration -> stop if $\|r_k\|$ is very small/after a maximum number of iterations.



The Conjugate Gradient method quickly settles on the exact solution around iteration 150, which matches its theoretical property of converging in at most n steps for an n -dimensional system in exact arithmetic. The dashed line is a simple linear bound based on the condition number of A , but it clearly overestimates how many iterations are actually needed.

Task 3.

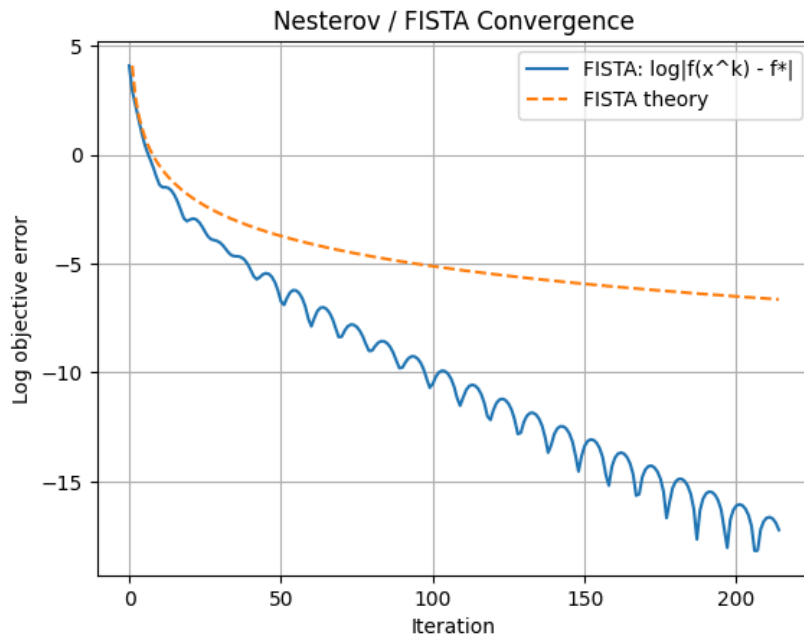
We keep track of:

$$t_{k+1} = \frac{1}{2} \left(1 + \sqrt{1 + 4t_k^2} \right)$$

$$x_{k+1} = y_k - \frac{1}{L} \nabla f(y_k)$$

$$y_{k+1} = x_{k+1} + \frac{t_k - 1}{t_{k+1}} (x_{k+1} - x_k)$$

Initialize $x_0 = 0$, $y_0 = x_0$, $t_0 = 1$ -> Perform updates for a fixed number of iterations or until the gradient is small enough.



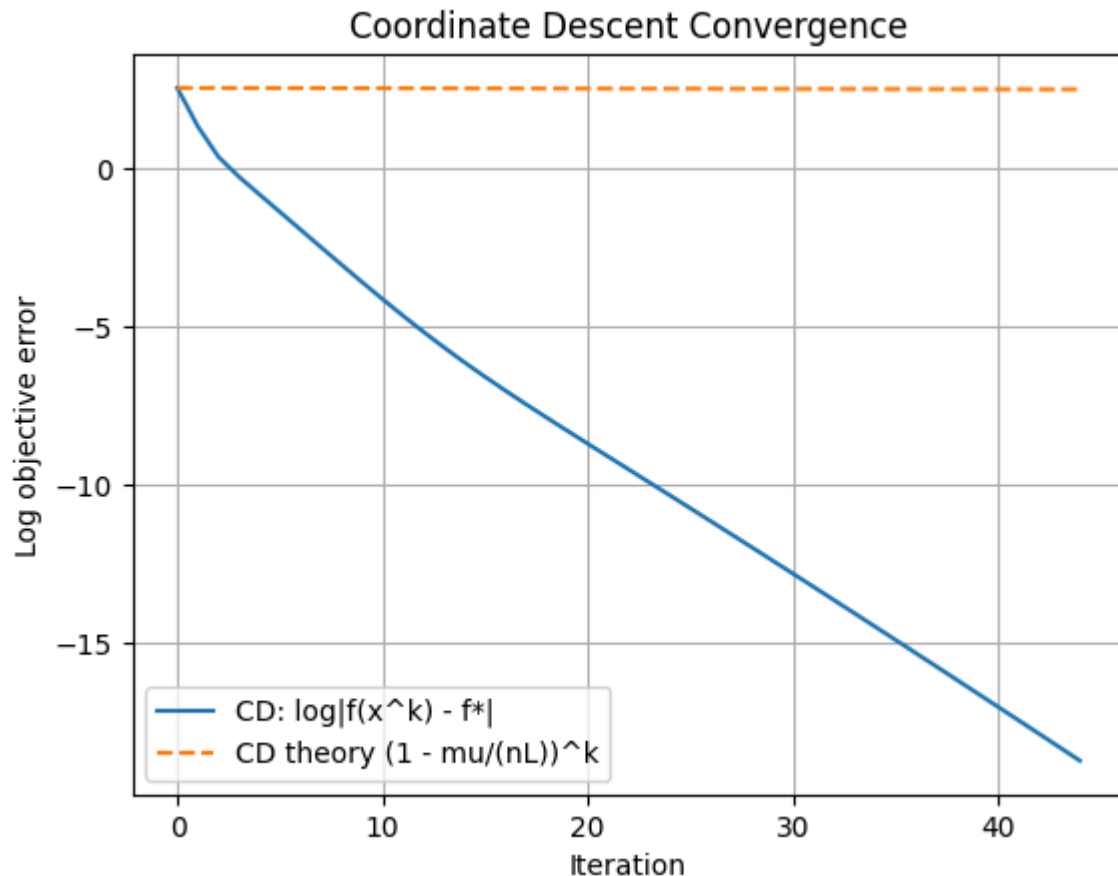
Around iteration 0, the FISTA method starts with a large error and decreases it over time. The dashed orange line shows the theoretical $\frac{1}{k^2}$ convergence, and we see that the actual FISTA convergence is generally in line with, though faster. This reflects the well-known accelerated performance of FISTA compared to standard gradient methods.

Task 4.

The code iterates through all the coordinates in a cyclic fashion and updates them each.

Rate: A simple bound for cyclic coordinate descent on strongly convex quadratics is

$$\left(1 - \frac{\mu}{nL}\right)^k$$



The plot shows that Coordinate Descent steadily decreases the objective error in a roughly linear fashion (straight line on the log-scale). The dotted line is the theoretical bound $\left(1 - \frac{\mu}{nL}\right)^k$, and we see the actual method converges faster than this bound predicts, which is typical of coordinate descent in practice.

Task 5.

1. Methods:
 - Direct Solve (np.linalg.solve)
 - Gradient Descent
 - Conjugate Gradient
 - FISTA
 - Coordinate Descent
2. Measurements:
 - Iterations until a stopping criterion (tolerance).
 - Wall-Clock Time using Python's time.time().

=== Dimension n = 100 ===

Backslash: time=0.0001s, final error=8.08e-02

ELEC-E5431 - Large Scale Data Analysis D
Bui Minh Anh Pham - 1024739
Homework assignment 1

GD: iters=72, time=0.0004s, final error=8.08e-02
CG: iters=33, time=0.0004s, final error=8.08e-02
FISTA: iters=198, time=0.0029s, final error=8.08e-02
CD: iters=46, time=0.0059s, final error=8.08e-02

=== Dimension n = 500 ===

Backslash: time=0.0030s, final error=1.02e+03
GD: iters=73, time=0.0028s, final error=1.02e+03
CG: iters=36, time=0.0015s, final error=1.02e+03
FISTA: iters=234, time=0.0168s, final error=1.02e+03
CD: iters=47, time=0.0257s, final error=1.02e+03

=== Dimension n = 1000 ===

Backslash: time=0.0185s, final error=2.50e+03
GD: iters=75, time=0.0091s, final error=2.50e+03
CG: iters=36, time=0.0042s, final error=2.50e+03
FISTA: iters=244, time=0.0574s, final error=2.50e+03
CD: iters=49, time=0.0640s, final error=2.50e+03

=== Dimension n = 2000 ===

Backslash: time=0.1153s, final error=5.21e+03
GD: iters=75, time=0.0503s, final error=5.21e+03
CG: iters=37, time=0.0271s, final error=5.21e+03
FISTA: iters=253, time=0.2992s, final error=5.21e+03
CD: iters=50, time=0.1926s, final error=5.21e+03

=== Dimension n = 5000 ===

Backslash: time=1.6009s, final error=1.38e+04
GD: iters=78, time=0.4360s, final error=1.38e+04
CG: iters=38, time=0.2056s, final error=1.38e+04
FISTA: iters=255, time=3.0065s, final error=1.38e+04
CD: iters=52, time=1.0191s, final error=1.38e+04

=== Dimension $n = 10000$ ===

Backslash: time=3.3592s, final error=2.72e+04

GD: iters=79, time=1.6594s, final error=2.72e+04

CG: iters=38, time=0.8157s, final error=2.72e+04

FISTA: iters=262, time=11.2634s, final error=2.72e+04

CD: iters=52, time=3.4094s, final error=2.72e+04

Observations:

Direct Solver:

Small Dimensions ($n = 100$): Extremely fast (0.0001 s). Gives the exact solution

Larger Dimensions ($n = 5000$ and 10000): Time grows significantly (1.60 s for $n = 5000$ and 3.36 s for $n = 10000$). Although still producing accurate results, the cost of performing a full matrix inversion increases roughly with the cube of the dimension. This is typical for dense direct methods.

Iterative Methods solve the system approximately by improving an initial guess through successive iterations. Their performance depends on the convergence rate, which in turn is affected by the condition number and structure of the matrix.

Gradient Descent (GD):

Around 72–79 iterations across all dimensions. Time scales with n (0.0004 s at $n = 100$ up to 1.66 s at $n = 10000$). GD's performance is competitive at small scales but is generally slower than more sophisticated methods as n increases.

Conjugate Gradient (CG):

Low iteration count (33–38 iterations even as n increases). Fastest among the iterative methods (e.g., 0.8157 s at $n = 10000$). CG leverages the symmetry and positive definiteness to converge in relatively few iterations, making it highly efficient for large-scale systems.

FISTA:

Requires a larger number of iterations (around 198–262). Significantly slower (up to 11.26 s for $n = 10000$). While FISTA is popular for problems with additional structure (like sparsity or non-smooth regularization), in this context it converges more slowly.

Coordinate Descent (CD):

Similar to GD for iteration count (around 46–52 iterations). Generally slower than CG, taking 3.41 s at $n = 10000$. CD can be appealing in certain high-dimensional settings or when the cost per coordinate update is very low, but here it is outperformed by CG.

Overall Comparisons and Conclusions

All methods eventually achieve the same “final error” for a given dimension, meaning they all converge to a similar quality solution.

Direct Solver:

- Exceptionally fast for small-scale problems ($n = 100$) but its time cost escalates quickly with dimension.
- For dimensions around $n = 5000$ – 10000 , the backslash operator takes several seconds, indicating that full inversion becomes computationally expensive.

Iterative Methods:

- CG stands out as the most efficient iterative method, with a nearly constant number of iterations and the smallest computation time as the problem scales.
- GD, FISTA, and CD show higher or comparable iteration counts, and their overall time increases more sharply compared to CG.

When Does the Problem Become “Large-Scale”?

- In the context of these experiments, while direct inversion (using backslash) remains feasible up to around $n = 10,000$ (taking on the order of 3–4 seconds), the relative benefit of iterative methods becomes apparent as the dimension increases.
- Once you reach dimensions where the direct method takes several seconds ($n \approx 5000$ and above), you start to see that iterative methods—especially CG—are more efficient. In very large-scale applications (n much larger than 10,000), direct inversion would likely be prohibitive both in time and memory, making iterative solvers the method of choice.

Problem 2

We want to find: $\text{prox}_f(y) = \left[f(x) + \frac{1}{2} \|x - y\|^2 \right]$, where $f(x) = g(ax + b)$

$$\text{Let } z = ax + b \rightarrow x = \frac{z-b}{a}$$

$$\text{Observe that } \|x - y\|^2 = \left\| \frac{z-b}{a} - y \right\|^2 = \frac{1}{a^2 \|z - (ay+b)\|^2}$$

Substitute $x = \frac{z-b}{a}$ into $f(x) = g(ax + b) = g(z)$, we have the min:

$$\left[g(z) + \frac{1}{2a^2} \|z - (ay + b)\|^2 \right]$$

But by the definition of the proximal operator with parameter

$$\lambda = a^2, \text{prox}_{a^2 g}(ay + b) = \arg\min [g(z) + \frac{1}{2a^2} \|z - (ay + b)\|^2]$$

$$\square \text{ Hence } z^* = \text{prox}_{a^2 g}(ay + b)$$

$$\text{Since } x = \frac{z-b}{a}, \text{ we get: } x^* = \frac{1}{a}(z^* - b) = \frac{1}{a}(\text{prox}_{a^2 g}(ay + b) - b)$$

$$\Rightarrow \text{Therefore, } \text{prox}_f(y) = \frac{1}{a}(\text{prox}_{a^2 g}(ay + b) - b)$$